
0.1 Trustless universal protocol for interoperable chains based on stateless clients

A high-level overview showing protocol flow can be seen in figure 1.

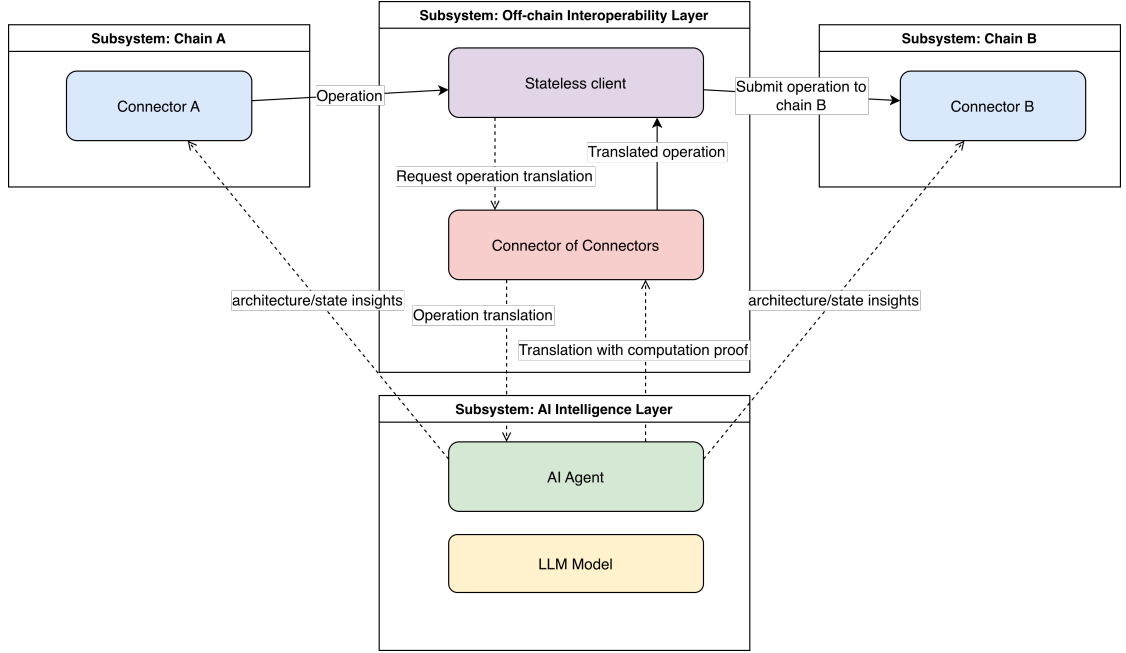


Figure 1: High-level protocol architecture overview.

The protocol consists of three main pillars that communicate with each other and with the chains:

- Smart contract connectors,
- Stateless clients,
- AI Agents.

The smart contract connectors are essentially middleman contracts that handle disputes, verify data, store data, and allow stateless clients to query information.

0.1.1 Smart contract connector

To create a trustless environment between chains, the smart contracts are designed to verify provided zero-knowledge proofs, and if they match the instruction that precedes the action they should perform, they perform it.

The design and simple example of operation of connector contracts can be observed in Figure 2.

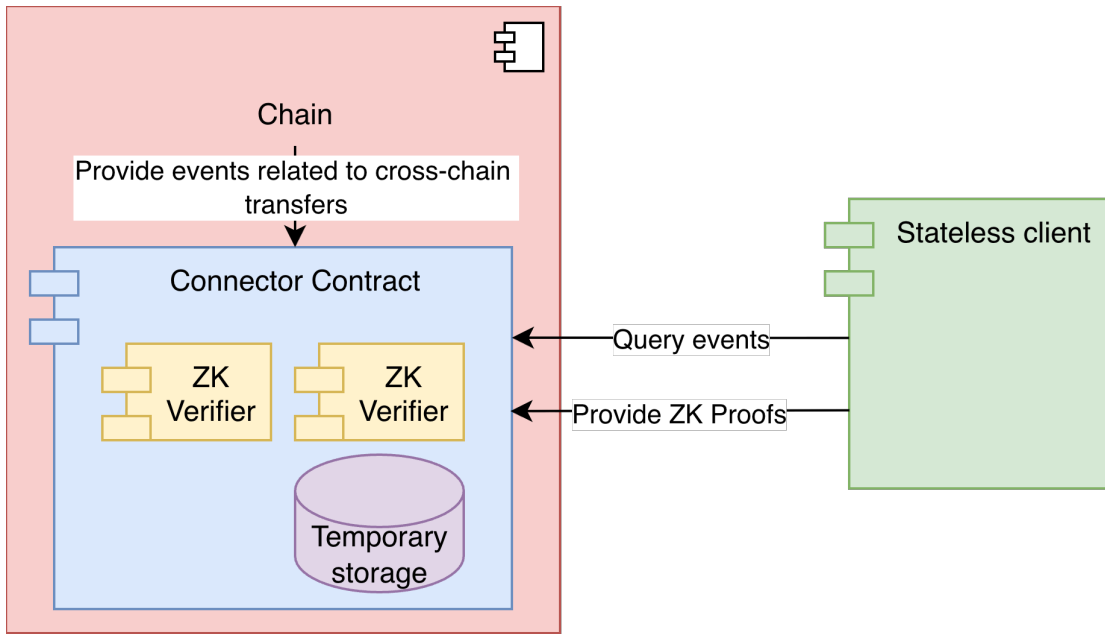


Figure 2: High-level overview of simple connector contract workflow.

As we can observe, the connector contracts handle chain information, and let stateless clients independently read all required data regarding their transaction. In addition, connector contracts verify submitted ZK proofs, manage the dispute process, and store both pending and executed transactions.

The communication system contains three cross-checks. This allows connectors to be certain that they can lock and release the funds. This system prevents double-spending when the user initiates a dispute. The system is illustrated in Figure

3.

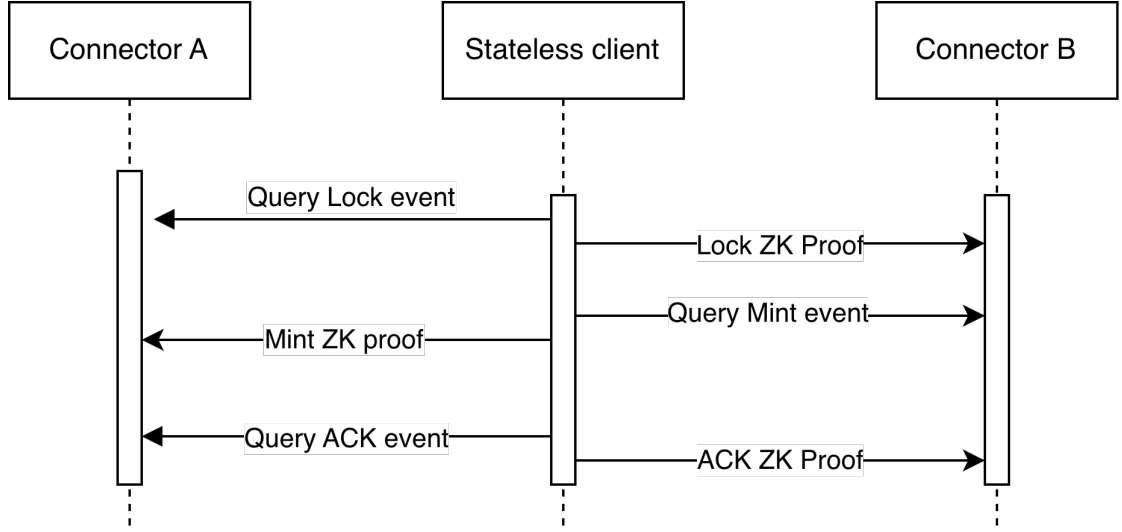


Figure 3: Cross-check system communication flow between connector contracts.

The transaction status is tracked within the smart contracts themselves, and communication is relayed through a Stateless Client that proactively informs both connectors. Successful transactions are removed from contracts immediately upon completion, whereas unsuccessful ones are removed only in the origin connectors. The destination connectors store unsuccessful transactions until the Stateless Client provides an ACK proof stored on the Origin chain. Origin does not need to store transactions after the mint proof is delivered, because the client can compute the ACK proof from the finishing event recorded on the blockchain directly.

If the receiving chain decides to reject the transfer, the protocol allows the user to refund the transaction after a certain period of time. This period must be long enough to allow for consensus and information exchange delays. The minted assets are temporarily held until the origin chain acknowledges that the mint on the destination chain happened and lets the destination chain know. If the origin connector never receives the mint proof, the assets are refundable after the deadline. The

refund process is illustrated in Figure 4 and is handled by the refund proof, which must be delivered to the destination chain. The destination chain then provides burn proof for the recently minted assets registry. Once these details are delivered, the refund is initiated. This process is necessary in order to remove transactions on the destination that would otherwise never finish and take up needed space in the smart contract. In case the Stateless client becomes unoperational at any point after delivering mint proof to the origin connector, the protocol allows it to submit the proofs at any point, finishing the transaction after the client's functionality is restored.

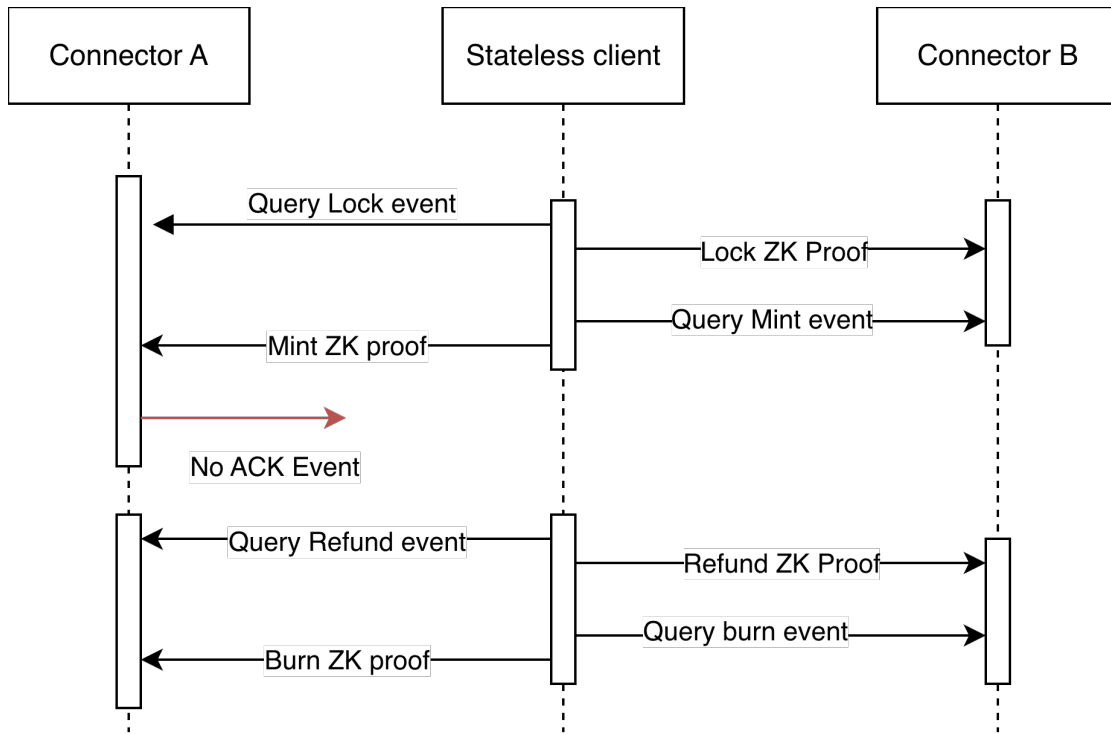


Figure 4: Refund case of the transaction.

0.1.2 Stateless client and zero-knowledge proofs

We have considered many different state-of-the-art approaches with both on and off-chain oracles at first [1, 2, 3, 4, 5, 6], but they all proved too resource-heavy, and not very cost-efficient to run when part of a cross-chain protocol.

Our next consideration was the usage of light clients [7], which do not need to know the full blockchain history, they only need block headers and cryptographic proofs. The downside of the light clients, however, is that they need constant synchronization with the blockchain, and they are much slower than traditional RPC clients or oracle networks [8].

We needed a client that does not rely on constant synchronization with the network, is light, and relatively fast. The optimal solution seems to be using a so-called "stateless client" [9]. These clients do not need to be synchronized with blockchains, as they are primarily designed to verify specific information without assumptions or context. They operate on demand, and with a bit of modification, we can make them relayers for origin and destination connector contracts. They will be initiated by the users, facilitating the cross-chain transactions. This has the added benefit of a free incentive to finish the relaying and not misbehave to prevent punishment, which also mitigates the need for third-party involvement. The proposed functionality of the stateless client in the proposed protocol is shown in Figure 5.

As depicted in the figure, the stateless client has multiple functions, such as:

- Querying transaction state from the connector,
- Querying zero-knowledge proofs from the chain in case of disconnect or other failure,
- Feeding connector contracts with zero knowledge proofs of lock events, mint

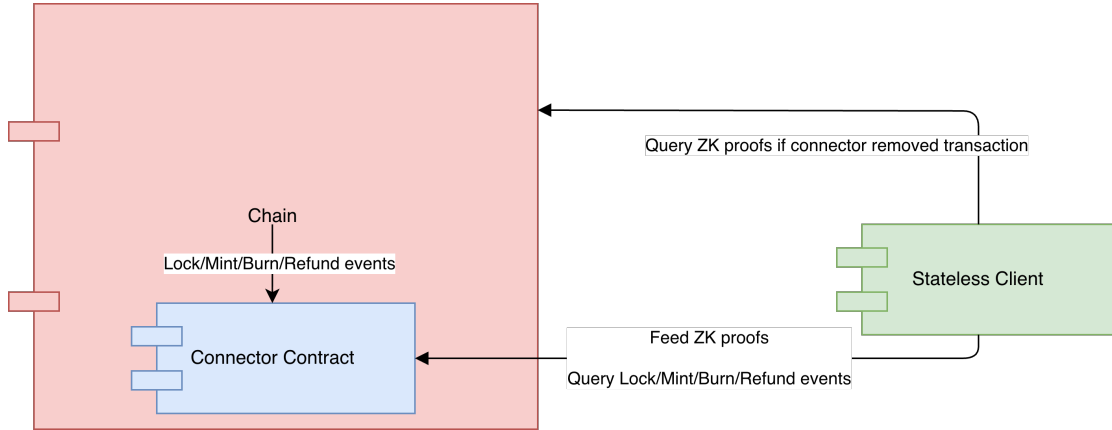


Figure 5: High-level overview of stateless client functionality.

events, or refund operation related events,

- Finding a source for off-chain zero-knowledge compute, or directly computing zero-knowledge proofs.

Regarding zero-knowledge proofs, we considered ZK-STARKs, which offer strong security advantages, including post-quantum resistance, and have been implemented in clients such as Zero-Sync [10]. However, they tend to require larger proof sizes and longer generation times, which can impact performance, price and storage requirements.

On the other hand, ZK-SNARKs provide faster proof generation and significantly smaller proof sizes, making them well-suited for less performant devices [11]. Historically, their main limitation has been the lack of post-quantum resistance. Nevertheless, recent research has proposed post-quantum-resistant ZK-SNARK verifier constructions [12], addressing this concern while preserving their performance benefits.

Zero-knowledge proofs can differ in terms of circuit definition, proving system, or verifier contracts [13]. Our first idea was to build a ZK-EVM translator, but

this proved impossible because ZK Proofs are one-way decryptable only, so we cannot decompose them to reconstruct them in another ZK-VM. However, our next approach is to allow different ZK-VMs by having multiple verifiers for the most widely used ZK-VMs on each connector.

0.1.3 Protocol flow

As outlined, the protocol design aims for simplicity while using a state-of-the-art approach through stateless clients and zero-knowledge proofs. The overall flow can be observed in Figure 6.

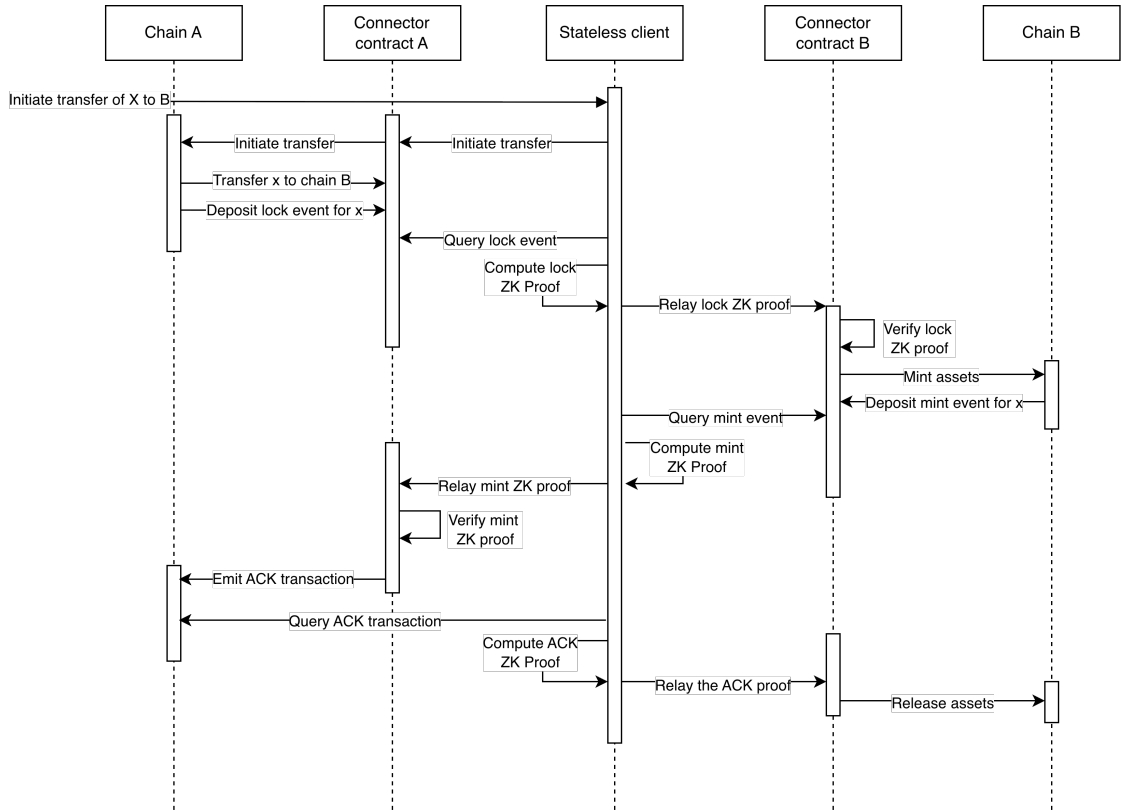


Figure 6: Sequential overview of an example protocol transaction flow.

After the user initiates the transaction, the origin chain notifies the connector contract and emits the deposit lock event. The connector contract then expects

the computation of a zero-knowledge proof, which will be provided by the stateless client. Once the proof is relayed to the destination connector, it gets verified. The destination connector notifies the destination chain to mint the assets. The mint is transferred into a temporary holding register, where the assets remain until the ACK proof is delivered. If the ACK proof does not arrive before the deadline, the assets remain locked until a refund claim from the origin chain is received.

After a valid refund claim, the assets are burned, and the stateless client is required to compute a burn zero-knowledge proof, which serves as the key for refunding the assets on the origin chain.

Once the destination chain completes the mint, it notifies its connector contract. The connector then waits for the stateless client to query the mint event and compute a zero-knowledge proof, which it relays to the origin connector. Once the mint proof is received, the dispute mechanism on the origin connector is disabled, and the only way to unlock the transaction funds is by relaying the ACK proof to the destination connector. The ACK proof can be computed from finishing the event happening on the origin connector after the mint proof is received. After the ACK proof is delivered, the destination connector instructs the destination chain to release the assets from the temporary holding register. Once this process is finished, the destination connector removes the cross-chain transaction data. The data on the origin connector is removed after the mint proof is delivered. This design incentivizes stateless clients to finalize transactions and helps clean the transaction pool.

Literature

- [1] Khalid Hassan, Amirreza Sokhankhosh, and Sara Rouhani. *Enabling Blockchain Interoperability Through Network Discovery Services*. 2025. arXiv: 2506.16611 [cs.DC]. URL: <https://arxiv.org/abs/2506.16611>.
- [2] Alessandro Bigiotti et al. “Smart Listeners: a Hybrid-Optimistic Inter-Blockchain Communication Protocol”. In: *Blockchain: Research and Applications* (2025), p. 100339. ISSN: 2096-7209. DOI: <https://doi.org/10.1016/j.bcra.2025.100339>. URL: <https://www.sciencedirect.com/science/article/pii/S2096720925000661>.
- [3] Alessandro Bigiotti et al. “Blockchain Agnostic Protocols: An Analysis of the State of the Art”. In: *ACM Comput. Surv.* (Aug. 2025). Just Accepted. ISSN: 0360-0300. DOI: 10.1145/3758089. URL: <https://doi.org/10.1145/3758089>.
- [4] Ghareeb Falazi et al. “Transactional Cross-Chain Smart Contract Invocations”. In: *Distrib. Ledger Technol.* 4.2 (June 2025). DOI: 10.1145/3616023. URL: <https://doi.org/10.1145/3616023>.
- [5] Michael Sober, Giulia Scaffino, and Stefan Schulte. “Cross-Blockchain Communication Using Oracles With an Off-Chain Aggregation Mechanism Based on zk-SNARKs”. In: *Distrib. Ledger Technol.* 3.4 (Dec. 2024). DOI: 10.1145/3678187. URL: <https://doi.org/10.1145/3678187>.

- [6] Tahrim Hossain et al. *CrossLink: A Decentralized Framework for Secure Cross-Chain Smart Contract Execution*. 2025. arXiv: 2504.09319 [cs.CR]. URL: <https://arxiv.org/abs/2504.09319>.
- [7] Panagiotis Chatzigiannis, Foteini Baldimtsi, and Konstantinos Chalkias. “Sok: Blockchain light clients”. In: *International Conference on Financial Cryptography and Data Security*. Springer. 2022, pp. 615–641.
- [8] Steffen Kux. *Stateless Light Clients — Enabling Trustless Blockchain Interaction (3 of 9)*. 2025. URL: <https://medium.com/corpus-ventures/stateless-light-clients-enabling-trustless-blockchain-interaction-3-of-9-934ffd0e050c>.
- [9] Jan Oberst. “Towards Stateless Clients in Ethereum: Benchmarking Verkle Trees and Binary Merkle Trees with SNARKs”. In: *arXiv:2504.14069* (2025).
- [10] Vishal Chawla. *ZeroSync implements first Stark-based ZK client for Bitcoin*. 2023. URL: <https://www.theblock.co/post/253876/zerosync-implements-first-stark-based-zk-client-for-bitcoin>.
- [11] Jan Oberst. *Towards Stateless Clients in Ethereum: Benchmarking Verkle Trees and Binary Merkle Trees with SNARKs*. 2025. arXiv: 2504.14069 [cs.CR]. URL: <https://arxiv.org/abs/2504.14069>.
- [12] Ken Naganuma et al. “Post-quantum zk-SNARKs from QAPs”. In: *International Journal of Information and Computer Security* 25.3-4 (2024), pp. 203–233.
- [13] Olivier Bégassat et al. “A specification for a ZK-EVM”. In: *Ethereum Res* (2021).